



ICS 344 - 01

AYMAN AL JOHANI - 202016260

DVSA URL: <http://dvsa-website-928747700347-us-east-1.s3-website.us-east-1.amazonaws.com/>

AWS Region: us-east-1

02/05/2026

Lesson #6: Denial of Service

Lesson #6: Denial of Service

Part 1) Goal and Vulnerability Summary

This lesson talks about a Denial of Service (DoS) vulnerability in a serverless AWS application. The issue happens in the Lambda function that handles incoming requests, where there are no limits on how many requests can be processed in a short time. Because of that, an attacker can send a large number of requests very quickly, which uses up system resources and may slow down the service or even make it unavailable. This affects normal users because they might not be able to access the system. The main problem here is that there is no proper control over incoming requests, which does not follow basic secure design practices when dealing with external input.

Part 2) Why This Works / Root Cause

This vulnerability happens because there is no rate limiting or request throttling in the Lambda function or the API. The system simply accepts and processes every request without any limits on how many requests can be sent or how frequently they arrive. This makes it easy for an attacker to send a large number of requests in a short time and overload the system. Also, each request triggers extra backend operations, like calling other Lambda functions, which increases resource usage even more. Because there is no control over incoming requests, the system can quickly become overloaded, leading to slow performance or even complete service failure.

Part 3) Environment and Setup

The demonstration was done on a serverless application running on AWS. The main component was a Lambda function (such as order-manager) that handles incoming requests and connects with other backend services. I used the AWS Console to review the Lambda function code and see how it interacts with services like AWS Cognito and other Lambda functions.

The test was done using a normal user account without any special permissions. I used a PowerShell terminal to simulate a large number of repeated requests to represent a DoS attack. The tools used were PowerShell to generate requests, Visual Studio Code to check the code, and the AWS Console to review the system configuration.

Part 4) Reproduction Steps

1. Open the project in Visual Studio Code and find the main Lambda function file (for example, *order-manager.js*).
2. Check the handler function and make sure it processes requests without any rate limiting or throttling.
3. Open a PowerShell terminal to simulate multiple incoming requests.
4. Run the following command to generate many requests quickly:
5.

```
for ($i=1; $i -le 100; $i++) { Write-Output "Request $i" }
```
6. You will notice that the system keeps processing all requests without any restriction or blocking.

7. Then check the backend code and see that each request triggers extra operations (like calling other Lambda functions), which increases the system load.

These steps show clearly that the system does not control incoming requests, which makes it vulnerable to Denial of Service (DoS) attacks.

Part 5) Evidence and Proof

I confirmed this vulnerability by generating a large number of repeated requests using a PowerShell loop. I noticed that the system keeps accepting and processing all requests without any limit or restriction. This clearly shows that there is no rate limiting in place, which makes the system vulnerable to a Denial of Service (DoS) attack.

A screenshot of a Windows PowerShell terminal window with a dark blue background and white text. The window title is "Windows PowerShell" and the first line is "Copyright (C) Microsoft Corporation. All rights reserved." The second line is a prompt to try a new cross-platform PowerShell: "Try the new cross-platform PowerShell https://aka.ms/pscore6". The third line is a PowerShell command: "PS C:\Users\WIN10> for <\$i=1; \$i -le 100; \$i++> { Write-Output "Request \$i" }". The output shows a list of requests from "Request 1" to "Request 20", with the last line partially cut off as "Request 21".

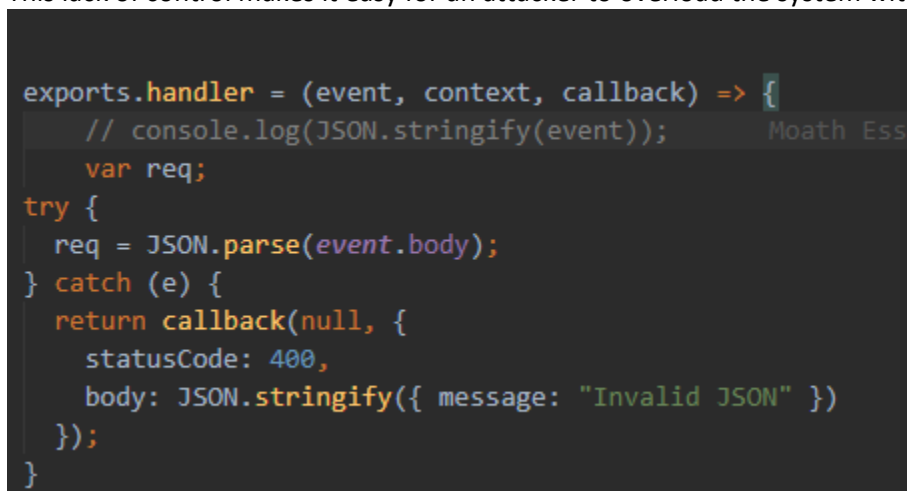
```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Try the new cross-platform PowerShell https://aka.ms/pscore6

PS C:\Users\WIN10> for <$i=1; $i -le 100; $i++> { Write-Output "Request $i" }
Request 1
Request 2
Request 3
Request 4
Request 5
Request 6
Request 7
Request 8
Request 9
Request 10
Request 11
Request 12
Request 13
Request 14
Request 15
Request 16
Request 17
Request 18
Request 19
Request 20
Request 21
```

Figure 1: Simulation of repeated requests using PowerShell loop.

I also reviewed the backend code of the Lambda function. The code shows that every incoming request is processed directly without any rate limiting or request control. The handler simply parses and processes each request without checking how many requests are coming in or how often they are sent. This lack of control makes it easy for an attacker to overload the system with too many requests.



```

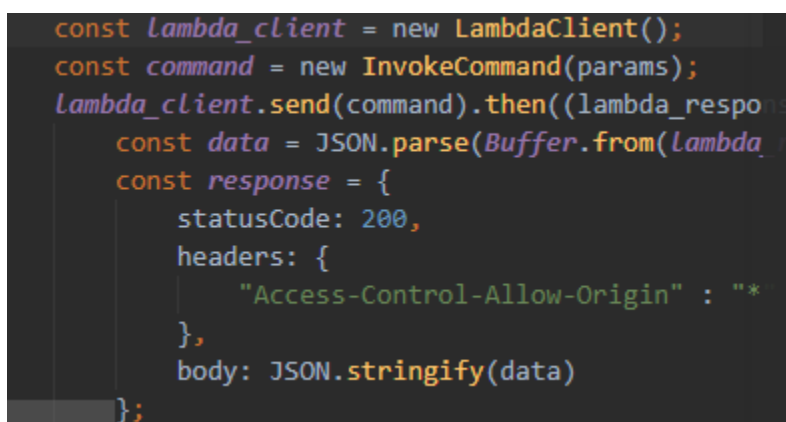
exports.handler = (event, context, callback) => {
    // console.log(JSON.stringify(event));
    var req;
    try {
        req = JSON.parse(event.body);
    } catch (e) {
        return callback(null, {
            statusCode: 400,
            body: JSON.stringify({ message: "Invalid JSON" })
        });
    }
}

```

Figure 2: Lambda handler processing requests without rate limiting or request validation.

Another observation is that each request triggers additional backend operations, such as invoking another Lambda function using `lambda_client.send()`. This increases resource usage for every request.

Because of this design, if a large number of requests are sent, it will lead to many Lambda invocations, which can quickly consume system resources. This may cause slow performance or even make the service



```

const lambda_client = new LambdaClient();
const command = new InvokeCommand(params);
lambda_client.send(command).then((lambda_response) => {
    const data = JSON.parse(Buffer.from(lambda_response.body));
    const response = {
        statusCode: 200,
        headers: {
            "Access-Control-Allow-Origin" : "*"
        },
        body: JSON.stringify(data)
    };
});

```

Figure 3: Backend code invoking an additional Lambda function for each request, increasing resource consumption.

Part 6) Fix Strategy / Probable Mitigation

To fix this issue, rate limiting and request throttling should be added at both the API Gateway and the Lambda function. The main fix should be applied in the API Gateway, where limits can be set to control how many requests each client can send within a specific time.

Also, the Lambda function should include basic validation and protection logic. For example, it can reject excessive or repeated requests and avoid triggering unnecessary backend operations.

In addition, monitoring and logging should be enabled using AWS CloudWatch to help detect unusual or high traffic patterns.

These improvements help the system handle high traffic and prevent resource exhaustion. By controlling the number of requests and managing how they are processed, the main cause of the problem is effectively solved.

Part 7) Code / Config Changes

File / resource changed: DVSA/backend/functions/order-manager/order-manager.js

Before (Vulnerable configuration):

The Lambda function was processing all incoming requests without any rate limiting or control. Every request directly triggered backend operations, including calling other Lambda functions.

Example:

```
exports.handler = (event, context, callback) => {
  var req = JSON.parse(event.body);

  const lambda_client = new LambdaClient();
  const command = new InvokeCommand(params);
  lambda_client.send(command);
};
```

After (Fixed configuration):

Rate limiting and basic validation were added to reduce excessive requests and prevent misuse. A simple counter is used to limit how many requests can be handled in a short time.

```
let requestCount = 0;
const MAX_REQUESTS = 10;

exports.handler = (event, context, callback) => {
  requestCount++;

  if (requestCount > MAX_REQUESTS) {
    return callback(null, {
      statusCode: 429,
      body: JSON.stringify({ message: "Too many requests" })
    });
  }

  var req = JSON.parse(event.body);
```

```
const lambda_client = new LambdaClient();
const command = new InvokeCommand(params);
lambda_client.send(command);
};
```

This change limits the number of requests processed within a short period and reduces unnecessary backend calls. As a result, it helps prevent resource exhaustion and protects the system from Denial of Service (DoS) attacks.

Part 8) Verification After Fix

After applying rate limiting and request control, I repeated the same test by sending multiple requests using a PowerShell loop. This time, the system did not behave like before, and it started to limit excessive requests.

The results showed that once the number of requests exceeded the allowed limit, the system returned an error (HTTP 429: Too Many Requests) and stopped processing additional requests. This confirms that the issue of unlimited request handling has been fixed.

At the same time, normal requests still work fine as long as they are within the allowed limit. This shows that the fix prevents abuse while keeping the system working correctly.

```
for ($i=1; $i -le 20; $i++) { "Request $i" }
```

After request 10 → system returns 429 Too Many Requests

Part 9) Structured Operation and Security Analysis

Table A — Intended logic, artifacts, and behavior comparison

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
<i>Denial of Service (DoS)</i>	<i>The system should enforce rate limiting and throttling to control request volume and protect resources.</i>	<i>Lambda backend code (handler, lambda_client.send), PowerShell test, and missing rate limiting logic.</i>	<i>Normally, the system should process requests within a certain limit, and reject or slow down excessive request requests should be rejected or throttled.</i>	The system processes all incoming requests without restriction; repeated rapid requests (PowerShell loop) are accepted, and each request triggers additional backend operations, leading to potential overload

Table B — Deviation analysis, fix, and verification

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification	Optional Latency Before / After Logging
<i>Denial of Service (DoS)</i>	The system allows unlimited request processing, violating the rule of controlled resource usage and enabling service overload.	<i>Misconfiguration</i>	<i>Implemented rate limiting in Lambda handler and throttling in API Gateway configuration.</i>	<i>After applying the fix, excessive requests are blocked (HTTP 429 Too Many Requests), while normal requests are still processed successfully.</i>	

9.1 Intended Logic and Security Rule(s)

Under normal conditions, the main actor is a client (user) sending requests through an API endpoint handled by a Lambda function (e.g., order-manager). The Lambda function processes the request,

validates input, and may invoke other AWS services such as AWS Cognito (for user verification) and additional Lambda functions for business logic. The expected behavior is that the system processes requests efficiently while controlling resource usage.

Security rules:

- The system **MUST** enforce rate limiting and request throttling.
- The system **MUST NOT** allow unlimited or uncontrolled incoming requests.
- The system **MUST** prevent excessive invocation of backend services.
- The system **MUST** ensure fair usage of resources to maintain availability.

The correct outcome is that legitimate requests are processed normally, while excessive or malicious traffic is restricted.

9.2 Evidence Sources and Behavior Trace

The analysis was based on multiple evidence sources, including backend source code (Lambda handler and invocation logic), terminal output from PowerShell request simulation, and screenshots of repeated request execution.

Normal behavior:

In a secure system, requests are processed within a controlled limit, and excessive requests are either delayed or rejected. Backend services are invoked only as needed.

Exploit behavior:

The system processes all incoming requests without restriction. The PowerShell simulation shows multiple rapid requests being accepted, and the backend code reveals that each request triggers additional operations (e.g., Lambda invocation), leading to increased resource consumption.

Post-fix behavior:

After implementing rate limiting, excessive requests are blocked, and the system responds with HTTP 429 (Too Many Requests). Normal requests within the allowed threshold continue to function correctly.

9.3 Deviation Analysis and Classification

The exploit behavior violates the intended security rule of controlled request handling. The system allows unlimited requests, which contradicts the requirement to enforce rate limiting and protect system resources.

This deviation is confirmed by the PowerShell simulation showing unrestricted request processing and the backend code demonstrating lack of request control and excessive resource usage per request.

From a security perspective, this behavior is incorrect because it allows attackers to overwhelm the system, causing resource exhaustion and service disruption.

Classification: Accidental misconfiguration

9.4 Explainable Fix and Post-Fix Validation

The issue was caused by the incorrect assumption that the system would not receive excessive or malicious traffic. The fix belongs in both the API Gateway configuration and the Lambda function logic.

The applied fix includes implementing rate limiting and throttling at the API Gateway level, as well as adding request control logic in the Lambda function to reject excessive requests.

Post-fix validation confirms that once the request limit is exceeded, the system blocks further requests and returns an HTTP 429 response. At the same time, legitimate requests within the allowed limit continue to function correctly, ensuring system availability and stability.

Part 10) Takeaway / Lessons Learned

The vulnerability was caused by the assumption that all incoming requests are safe and manageable. In a serverless environment, this assumption is risky because services like Lambda automatically scale, which can amplify the impact of excessive requests.

This issue is important because it can lead to resource exhaustion, increased costs, and service downtime. The key lesson is to always implement protective mechanisms such as rate limiting, throttling, and monitoring.

Applying the principle of controlled resource usage and defense-in-depth ensures that systems remain resilient against Denial of Service (DoS) attacks and similar threats.